

Exam Introduction to Databases

2 February 2018

- The final exam contains 5 problems and is scheduled for 4 hours.
- Please verify if your question booklet consists of 6 pages (plus 7 pages with supplementary lecture slides) that are printed legibly, else contact your tutor immediately.
- The exam questions address both the theory and exercise parts of the lecture. For questions marked with an asterisk (*), *supplementary lecture* slides are provided at the end of this booklet.
- No electronic devices (such as notebooks, smartphones or smartwatches) are allowed.
- Usage of course material is not allowed.
- Answers *without sufficient details* are *void* (e.g.: you can't just answer "yes" or "no").
- Only the answer booklet will be collected, please do not use the question booklet for answering the questions.
- Take pride in your work: answer clearly and hand in a readable exam copy!

Problem 1: Entity Relational Model and Relational Design (5 points)

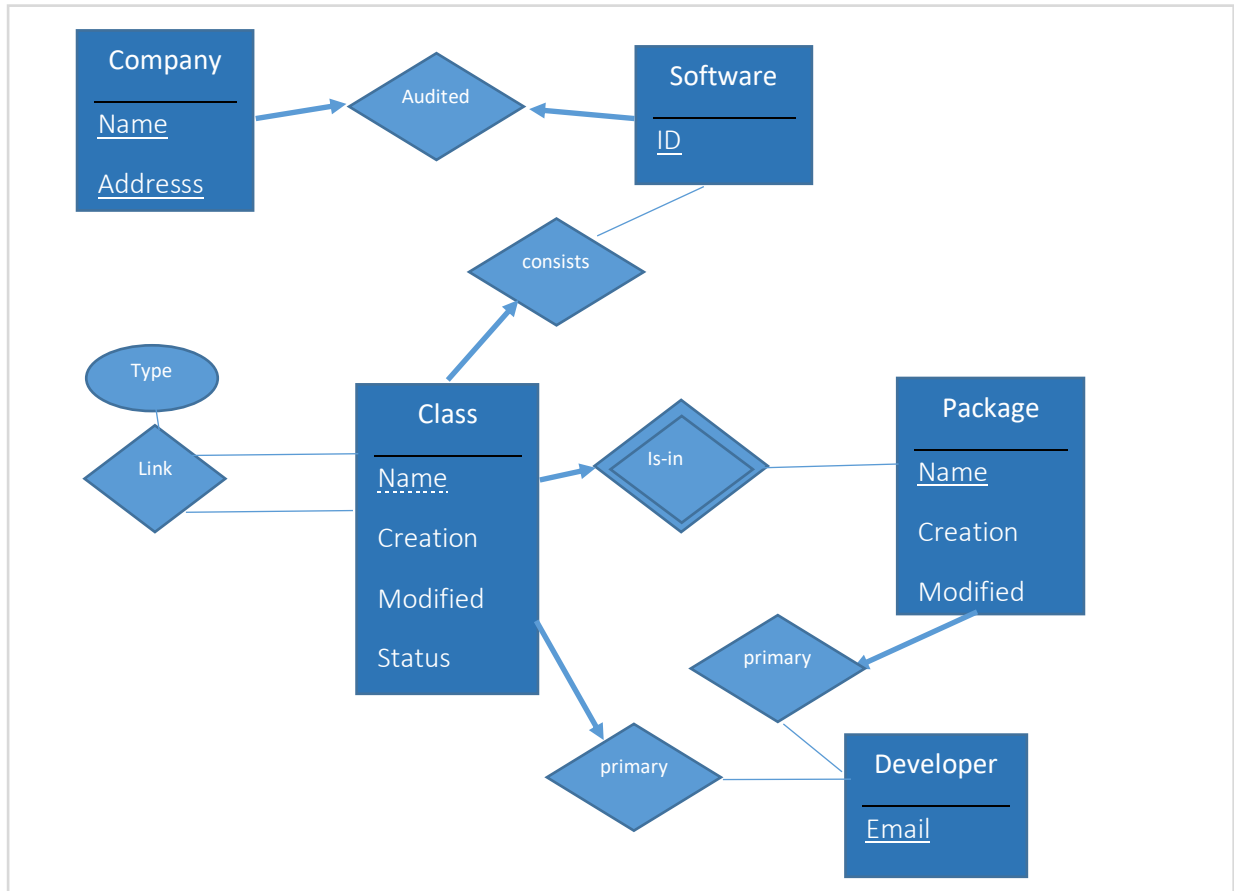
Carefully read the following requirement analysis.

A consultancy firm is specialized in auditing software systems for companies. Their service is to inspect the code and find potential bugs. To support this service, they need a database that can model the structure of the different software systems developed at each company. Each company has a name and an address, and is represented uniquely by the combination of both name and address. Each company has exactly one software system that is audited by the consultancy firm. A software system consists of one or more classes. Related classes are grouped into a single package (or namespace). A class name is unique by combining it with the name of its package. A class and a package both have a name, a creation date, a modified date and the primary developer. A developer has a unique e-mail address. A class also has an audit status which is either "to-inspect", "verified-ok" or "verified-not-ok". Finally, there are links between classes. The link type is either "dependency" or "extension" in case of inheritance.

1. Create an Entity-Relationship diagram that models as faithfully as possible the above description. Clearly indicate entities, relationships, primary keys, cardinalities and participation constraints.
2. (*) Create the necessary create table statements to implement this ER-diagram as a relational database. Define all necessary schema-level constraints, such as primary keys, foreign keys and domain constraints. Active database instructions such as "On delete/update cascade/..." are not required.
3. Create a check to assert that the modified date of a package, is always higher or equal than the modified date of each class in that package.
4. Create a check to assert that there are no cyclic dependencies between classes, that is, if A depends on B, B should not depend on A.
5. Create a check to assert that there are no cyclic dependencies between classes, but also consider indirect links, e.g. if A depends on B, and B depends on C, and C depends on A, then B also, indirectly depends on A, and there's a cycle. You can assume that at most 2 links connect (or don't connect) any two classes, and ignore the more general case of at most N links.

Sample Solution

1.



- Assumption: Every class belongs to 1 software system.
- Each company has 1 audited software system.
- Link is a many-to-many relation between two classes. We assume this relation is not mandatory. Type is added as attribute to the relation. An alternative is to specify one relation for dependency and one for extension.
- As class is identified by the combination of both package name and class name, thus class becomes a weak entity, with name as partial key.
- Inheritance between class and package and something like "SoftwareEntity" is possible, but then expressing the constraint that that class name is unique by combining it with package name, is easy to get wrong. It is possible, but then name should not be defined in SoftwareEntity.
- Remark that adding attributes such as developer_email, package_name or software_id to another entity, instead of using a relation, is correct in SQL, but *incorrect* in E.R. E.R. is a high-level conceptual notation that summarizes relations between entities. In SQL tables are actual tables and can correspond to either entities, relations or something-in-between.

2.

```
CREATE table Company{
    name VARCHAR,
    address VARCHAR,
    PRIMARY KEY (name,address)
};

//Note the definition of the composite key (name,address).
```

```
CREATE table Software{
    id INT PRIMARY KEY,
    company_name VARCHAR,
    company_address VARCHAR,
    FOREIGN key (company_name, company_address) REFERENCES Company(name,address)
};
```

//Note the notation to define a foreign key to an entity that has a composite key. An alternative solution, since this is a one-to-one relation, is to add an attribute software in company. If you define both attributes, however, this is, unwanted, redundancy.

```
CREATE DOMAIN AuditStatus VARCHAR CHECK VALUE in ('to-inspect', 'verified-ok', 'verified-not-ok');
```

```
CREATE table Class{
    name VARCHAR,
    package_name VARCHAR,
    modified DATETIME NOT NULL,
    creation DATETIME NOT NULL,
    status AuditStatus NOT NULL,
    developer_email VARCHAR NOT NULL,
    software_id INT NOT NULL,
    PRIMARY KEY (name, package_name),
    FOREIGN KEY package_name REFERENCES Package(name),
    FOREIGN KEY developer_email REFERENCES Developer(email),
    FOREIGN KEY software_id REFERENCES Software(id)
} ;
```

//Note that one-to-many relations are translated to an additional attribute with a foreign key. Since these relations are mandatory the NOT NULL constraint is added. Also not that all edges defined in E.R. must correspond with a foreign key in SQL. In this case, class plays a role in three relations: Each class tuple has exactly one developer ("primary"), and a link to exactly 1 software system ("consisting"), and exactly one link to the package ("is-in"). Link is a many-to-many relation and needs its own table in SQL (with at least two foreign keys).

```

CREATE table Package{
    name VARCHAR,
    modified DATETIME NOT NULL,
    creation DATETIME NOT NULL,
    developer_email VARCHAR NOT NULL,
    PRIMARY KEY (name),
    FOREIGN KEY developer_email REFERENCES Developer(email),
} ;

CREATE table Developer{
    email VARCHAR PRIMARY KEY
};

CREATE DOMAIN LinkType VARCHAR CHECK VALUE in ('dependency', 'extension');

CREATE table Link{
    class1_name VARCHAR,
    package1_name VARCHAR,
    class2_name VARCHAR,
    package2_name VARCHAR,
    type LinkType NOT NULL,
    PRIMARY KEY(class1_name, package1_name, class2_name, package2_name),
    FOREIGN KEY (class1_name, package1_name) REFERENCES Class(name,package_name),
    FOREIGN KEY (class2_name, package2_name) REFERENCES Class(name,package_name),
};

//Since class is uniquely identified by the composite key (name, package_name), we
must also include both attributes and specify the foreign key on the composite key.
Note that other variations are mostly incorrect, but one option is to add a primary
key ID to class, and specify a unique and null constraint on the composite key
(name, package_name) inside class. This way we can use class.id as non-composite
foreign key.

```

3.

```
CREATE ASSERTION check_modified AS
NOT EXISTS ( SELECT *
              FROM Package, Class
              WHERE Package.name = Class.package_name
              AND class.modified > package.modified);
```

4.

```
CREATE ASSERTION check_cycles_simple AS -- (e.g. (a,b) and (b,a))
NOT EXISTS ( SELECT *
              FROM Link l1, Link l2
              WHERE l1.class1_name = l2.class2_name
                  and l1.package1_name = l2.package2_name
                  and l1.class2_name = l2.class1_name
                  and l1.package2_name = l2.package1_name
                  and l1.type = 'dependency'
                  and l2.type = 'dependency'
              );
```

Note the use of a cartesian product here.

5.

```
CREATE VIEW LinkSimple AS
SELECT class1_name || '-' || package1_name as from_class,
       class2_name || '-' || package2_name as to_class
FROM Link
WHERE type = 'dependency'
```

//We add a view LinkSimple to simplify the main query. Using LinkSimple we can compare class identity with 1 compare, instead of checking both package_name and class_name.

```
CREATE ASSERTION check_cycles_link2 AS -- (e.g. (a,b) and (b,c),(c,a)
NOT EXISTS (SELECT *
              FROM LinkSimple l1, LinkSimple l2, LinkSimple l3
              WHERE  l1.to_class = l2.from_class
                  and l2.to_class = l3.from_class
                  and l3.to_class = l1.from_class)
```

Problem 2: Triggers, SQL and Relational Algebra (5 points)

Consider the following relational schema:

Customer

cid	street	streetnumber	City
1	Middelheim laan	1	Antwerp
2	Grote steenweg	203	Antwerp

ElectricityMeter

mid	cid	total_use (kwh)	modified
1001	1	100 000	2018-02-02 10:00
1002	2	50 101	2018-02-02 09:00
1003	1	78 888	2018-02-02 10:00

Sensor

sid	mid	total_use (kwh)	Description
s1	1001	80 000	Offices
s2	1001	20 000	Lights
s3	1002	50 101	General
s4	1003	12 483	Lights

SensorLog

sid	Datetime	use (kwh)
s1	2018-02-02 10:00	11.1
s1	2018-02-02 09:00	9.8
s1	2018-02-02 08:00	5.0
s1	...	...
s2	2018-02-02 10:00	0.41
s2	2018-02-02 09:00	0.89
s2	...	...

- (*) Write a trigger that assures that the corresponding values of ElectricityMeter.total_use and Sensor.total_use are updated, every time a new row is added to SensorLog.
- Write the following queries in SQL:
 - Return a list of customers and sensor descriptions, where the sensor has the *highest* total_use.
 - Return the customer in Antwerp which had the *highest* usage during January 2018. Note that historical usage is only stored in the sensor logs.
Tip: you can use *Extract(month from COLUMN)*, *Extract(year from COLUMN)*, etc. to extract parts of the time information.
 - List the customers addresses in Antwerp where *all* sensors with description "lights" have *no* power consumption (use is 0.0) for any night in January 2018 between 00:00 and 06:00.
 - List pairs of electricity meters that have a different id, but the same values for total_use for *all* corresponding sensors.
- (*) Write the following queries in Relational Algebra:
 - Return a list of customers and sensor descriptions, where the total use of the sensor is 0.0
 - Return *all* customers in Antwerp that do *not* have *any* sensor for lights.

Sample Solution

1. Trigger:

```
CREATE TRIGGER UPDATE_TOTAL AFTER INSERT ON TABLE SensorLog REFERENCING new row
as new_row
FOR EACH ROW
BEGIN
    UPDATE Sensor
    SET total_use = total_use + new_row.value
    WHERE Sensor.sid = new_row.sid;

    UPDATE ElectricityMeter
    SET total_use = total_use + new_row.value
    FROM ElectricityMeter, Sensor
    WHERE Sensor.sid = new_row.sid and ElectricityMeter.mid = Sensor.mid;

END;
```

Note that we need to update two tables, and that for updating ElectricityMeter an extra join is needed, as MID is not in SensorLog and only in sensor. Correct syntax is vendors specific for triggers, but update statements are always the same and the extra join is always necessary.

2. SQL:

a.

Many possible solutions exist, such as:

```
SELECT distinct c.cid, s1.description
FROM Customer c, ElectricityMeter m1, Sensor s1
WHERE c.cid = m1.cid and s1.mid = m1.mid
      AND total_use >=  SELECT max(s1.total_use)
                        FROM ElectricityMeter m1 JOIN Sensor s1 on m1.mid=s1.mid
                        WHERE m1.cid = c.cid
```

This is also correct:

```
CREATE VIEW max_use_view as
SELECT c.cid, max(s1.total_use) as maxi
FROM Customer c NATURAL JOIN ElectricityMeter m1 JOIN Sensor s1 on m1.mid = s1.mid
GROUP BY c.cid;
```

```
SELECT cid, description
FROM max_use_view JOIN ElectricityMeter m on max_use_view.cid = m.cid
      JOIN Sensor s on m.mid = s.mid
WHERE s.total_use = maxi;
```

However, `SELECT cid, description, max(total_use),` is wrong, since "max" is an aggregate function that is computed over all rows *grouped by* cid, and description is not defined in such a grouped setting.

b.

```
SELECT t1.id, max(t1.sum_customer)
FROM (SELECT c.id, sum(l. use) as sum_customer
      FROM Customer c, ElectricityMeter m, Sensor s, SensorLog l
      WHERE c.cid = m.cid and m.mid = s.mid  and s.sid = l.sid and
            EXTRACT(month from l.datetime) = 1 and
            EXTRACT(year from l.datetime) = 2018
      GROUP BY c.id) as t1
```

Note that writing "max(avg(column))" is in SQL. It is also unclear, to an interpreter, what the meaning would be of such an expression. The inner query computes the aggregate function "sum" based on all sensor values in January attached to each customer. The max function is thus computed over a grouping of a single customer. The "max" function, however, is computed, over a single sum for each customer. Thus, SQL requires some form of nested or subquery to compute this result.

c.

```
SELECT c.street, c.streetnumber
```

```

FROM Customer c
WHERE c.city = "Antwerp" and NOT EXISTS(
    SELECT *
    FROM ElectricityMeter m, Sensor s, SensorLog l
    WHERE m.cid = c.cid and s.mid = m.mid and l.sid = s.sid
        and s.description = "lights" and
        EXTRACT(month from l.datetime) = 1 and
        EXTRACT(year from l.datetime) = 2018 and
        EXTRACT(hour from l.datetime) >= 0 and
        EXTRACT(hour from l.datetime) <= 6 and
        value > 0.0
)

```

Like the solution to question 1, the "not exists" or $\forall r1 \in R : \nexists r2 \in R : predicate(r1, r2)$ way of filtering is crucial. Otherwise we would return customers that may only had value=0 for lights at a single timepoint during the night, and not all nights in January. The following alternative solution is also correct:

```

(SELECT c.street, c.streetnumber
FROM Customer c
WHERE c.city = "Antwerp")
EXCEPT
(SELECT c.street, c.streetnumber
FROM Customer c, ElectricityMeter m, Sensor s, SensorLog l
WHERE m.cid = c.cid and s.mid = m.mid and l.sid = s.sid
    and s.description = "lights" and
    EXTRACT(month from l.datetime) = 1 and EXTRACT(year from l.datetime) = 2018
    and EXTRACT(hour from l.datetime) >= 0
    and EXTRACT(hour from l.datetime) <= 6 and value > 0.0)

```

d.

```

SELECT m1.mid, m2.mid
FROM ElectricityMeter m1, ElectricityMeter m2
WHERE m1.mid != m2.mid and NOT EXISTS(
    SELECT *
    FROM Sensor s1, Sensor s2
    WHERE s1.total_use != s2.total_use
        and s1.mid = m1.mid and s2.mid = m2.mid)

```

This is true, since we return every pair of meters with distinct id's, where it is not the case that any total_use value is different. Like the questions before, a lot of mistakes against logic where submitted.

3. R.A:

a.

$\pi_{cid, description}(customer \bowtie_{customer.cid=meter.cid} meter \bowtie_{meter.mid=sensor.mid} \sigma_{totaluse=0}(sensor))$

b.

$$\begin{aligned}
& \pi_{cid}(\sigma_{city="Antwerp"} customer) \\
& - \pi_{cid}(customer \\
& \bowtie_{customer.cid=meter.cid} meter \bowtie_{meter.mid=sensor.mid} \sigma_{description="lights"}(sensor))
\end{aligned}$$

Problem 3: Database design (5 points)

Consider a database relation *Grades* with attributes *Student*, *Course*, *ExamDate*, *Mark*, *Program*, *Lecturer*, *Quarter*, *academic Year*. A tuple (s, c, e, m, p, l, q, y) in this relation expresses that student s followed course c in academic year y and quarter q as part of the program p . s received mark m in the exam that took place on date e . l is one of the lecturers of the course c in academic year y .

Example:

student	course	examdate	mark	program	lecturer	quarter	year
Peter	Databanken	2/2/2018	17	Ba INF	Toon	Q1	17-18
Peter	Databanken	2/2/2018	17	Ba INF	Len	Q1	17-18
Eva	Programmeren	19/9/2017	13	Ba INF	Jan	Q2	16-17

Peter took the course Databanken in quarter 1 of the academic year 2017-2018. The course was lectured by Len and Toon and he obtained 17 on the exam that was held on 2/2/2018. The course was followed as part of the program Ba INF. In the academic year 16-17, Eva followed the course Programmeren lectured by Jan in quarter 2. She obtained 13 on the exam which was held on 19/9/2017.

In this relation, the following rules hold:

1. A course will be offered multiple times, for instance yearly. We call a combination of a course and the quarter and academic year in which the course is given, a course offering. In every academic year there is at most one course offering per course.
2. A course can have multiple examination dates per course offering, but every examination date and course correspond to exactly one course offering.
3. With every course offering one or more lecturers are associated.
4. For every student only the last result for a course is stored. Hence, for every student and course, at most one examination date and mark is stored.
5. A student can be registered for at most one program per academic year.

Question:

- a. (*) Translate the rules 1-5 as a set of functional and multi-valued dependencies
- b. (*) Give a lossless decomposition of the relation in Boyce-Codd Normal Form.
- c. (*) Give a lossless and dependency-preserving decomposition of the relation in 3rd Normal Form.
- d. Give a lossless decomposition of the relation in 4th Normal Form.

Sample Solution

a.

1. $YC \rightarrow Q$
2. $EC \rightarrow QY$
3. $YCQ \rightarrow \rightarrow L$
4. $SC \rightarrow EM$
5. $SY \rightarrow P$

b. First we determine candidate keys:

S, C, and L need to be in any candidate key as they do not appear on the right-hand side of any FD. As they together determine all other attributes, SCL is the only candidate key.

FD $YC \rightarrow Q$ violates BCNF. $YC^+ = YCQ$ in R

$R_1 = YCQ$ $R_2 = SCEMPY$

R_1 is in BCNF. R_2 is not; $EC \rightarrow Y$ violates BCNF in R_2 . $EC^+ = ECY$ in R_2

$R_{21} = ECY$ $R_{22} = SCEMPL$

R_{21} is in BCNF, R_{22} isn't because of FD $SC \rightarrow EM$. $SC^+ = SCEMP$ in R_{22} .

$R_{221} = SCEMP$ $R_{222} = SCL$

R_{222} is in BCNF. R_{221} too.

Decomposition in BCNF: R_1 , R_{21} , R_{221} , and R_{222} .

c. The decomposition in a. is not dependency preserving because of $SY \rightarrow P$. First we minimize the FDs 1,2,4,5. From 2 we can remove Q from the right-hand side. This is the only attribute we can remove in any of the FDs. This gives us the following decomposition:

YCQ , ECY , $SCEM$, SYP , **SCL**

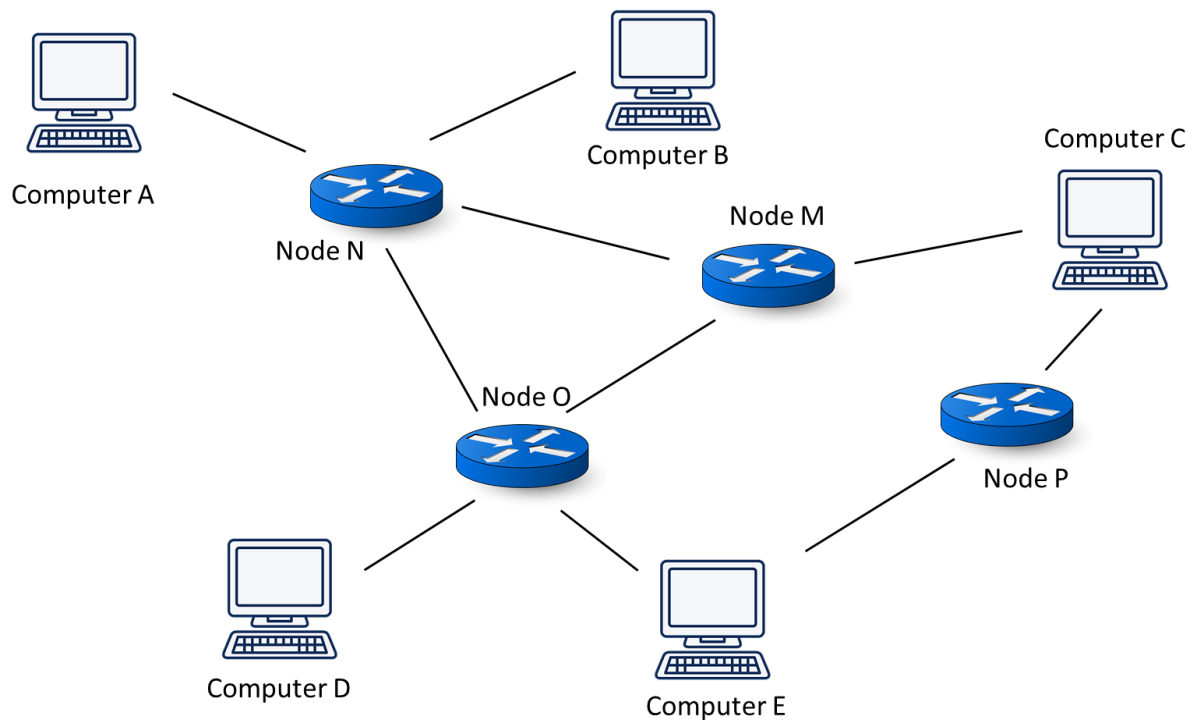
We had to add a relation containing a key, so hence SCL.

Notice that it was important to remove Q from $EC \rightarrow QY$, as otherwise the relation $ECQY$ contains the FD $YC \rightarrow Q$.

d. We can continue the decomposition of b. There is a MVD $YCQ \rightarrow \rightarrow L$ which leads to the MVD $SC \rightarrow \rightarrow L$ in R_{22} ($SC \rightarrow EM + EC \rightarrow QY$ implies $SC \rightarrow QYC$, with $QYC \rightarrow \rightarrow L$, this gives $SC \rightarrow \rightarrow L$). This is however a trivial MVD. Neither $C \rightarrow \rightarrow L$ nor $S \rightarrow \rightarrow L$ holds, so b. is already in 4NF.

Problem 4: Datalog (3 points)

Consider the following extensional database relation: $\text{Node}(x)$, $\text{Computer}(c)$, $\text{Connected}(x,y)$. $\text{Node}(n)$ indicates that n is a node (router) in a computer network, $\text{Computer}(c)$ that c is a computer in the network, and $\text{Connected}(x,y)$ that node or computer x is connected to node or computer y . For instance, the following network:



Is represented by the following relations:

Computer
A
B
C
D
E

Node
N
M
O
P

Connected	
A	N
N	B
N	M
N	O
O	M
M	C
P	C
P	E
O	D
O	E

(*) Write datalog rules that define the following intentional relations:

- $\text{reachable}(C1,C2)$: there is a path from computer $c1$ to computer $c2$ through the network. Keep in mind that if $\text{Connected}(n,m)$, then there is implicitly also a connection from m to n .
- $\text{bottleneck}(N)$: if we would remove node N from the network, then at least one reachable pair of computers is no longer reachable.
- Is your program stratified? Does your program always have a minimal model? Explain your answer!

Sample Solution

a.

```
consym(X,Y) :- connected(X,Y).  
consym(X,Y) :- connected(Y,X).  
reachable(X,Y) :- consym(X,Y).  
reachable(X,Z) :- reachable(X,Y), reachable(Y,Z).
```

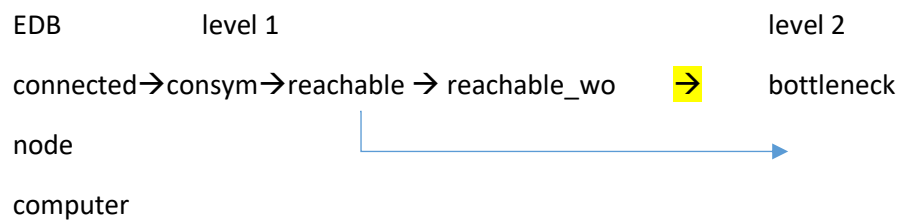
b.

```
consym(X,Y) :- connected(X,Y).  
consym(X,Y) :- connected(Y,X).  
reachable(X,Y) :- consym(X,Y).  
reachable(X,Z) :- reachable(X,Y), reachable(Y,Z).  
reachable_wo(X,Y,N) :- consym(X,Y), X<>N, Y<>N.  
reachable_wo(X,Z,N) :- reachable_wo(X,Y,N), reachable_wo(Y,Z,N).  
bottleneck(N) :- reachable(X,Y), node(N), not(reachable_wo(X,Y,N)).
```

Important: node(N) in bottleneck, otherwise not safe.

c.

Yes; the program is stratified:



So we can always compute the stratified model, which is a minimal model.

Problem 5: XML (2 points)

- a. Describe how you would store the data in Problem 3 (grades table) into an XML document.
- b. (*) Based on the structure of your document, give *XPath queries* for the following problems:
 - i. Give all students who obtained a grade of more than 10 for the course databases
 - ii. Give all course offerings for which “Paredaens” is one of the lecturers and for which at least 2 students obtained 20/20.

Sample solution

I would store a document with exams, grouped by offering. Every offering has course with its quarter and year and exam elements for each student, with mark and examdate. Course has lecturers inside. Student has program inside.

For instance:

```
<grades>
  <block quarter="Q1", year="97-98">
    <course name="databanken">
      <lecturer name="Paredaens"/>
      <lecturer name="Dekeyser"/>
      <exam date="21/1/1997">
        <student name="Toon">
          <program name="Ba INF" />
        </student>
        <mark>17</mark>
      </exam>
    </course>
  </block>
</grades>
```

- i. `//exam[mark>10]/student`
- ii. `//course[lecturer/@name="Paredaens"][count(exam[mark=20])>=2]`

Important here is how you organize the XML document; query ii can become vastly more difficult if the structure is poorly chosen

Defining Foreign Keys

```
CREATE TABLE Orders
```

```
(  
    O_Id int NOT NULL,  
    OrderNo int NOT NULL,  
    P_Id int,  
    PRIMARY KEY (O_Id),  
    FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)  
)
```

Does not have to be the same attribute name. But number and types need to match

Could also be multiple attributes

Enforces that every Orders.P_Id occurs in the Persons table in the column P_Id.



Assertions

(not supported in Oracle/Postgres, uses CREATE TRIGGER instead)

MovieExec(name, address, certN, netWorth)
Studio(name, address, presCertN)

- No one may be the president of a studio, unless this person has a net worth of at least \$1M :

```
CREATE ASSERTION RichPres CHECK  
(NOT EXISTS (  
    SELECT * FROM MovieExec, Studio  
    WHERE certN = presCertN AND netWorth < 1000000));
```

- Every executive name has only one address :

```
CREATE ASSERTION FunctionalDependency CHECK  
(NOT EXISTS (  
    SELECT * FROM MovieExec mv1, MovieExec mv2  
    WHERE mv1.name = mv2.name AND mv1.address <> mv2.address));
```



Triggers in SQL

General syntax:

```
CREATE TRIGGER <trigger_name>
{BEFORE|AFTER} {{INSERT|DELETE|UPDATE} [OF <row_list>]}* ON <table_name>
[REFERENCING [NEW ROW|TABLE AS <new_name>] [OLD ROW|TABLE AS <old_name>]]
[FOR EACH ROW|STATEMENT [WHEN (<trigger_condition>)]]
BEGIN
<trigger_body>
END;
```

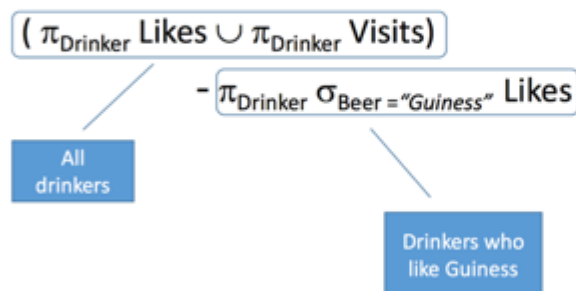


* SQL'99 allows both ROW and TABLE/STATEMENT level triggers.
Oracle allows only ROW level, hence the keyword ROW is omitted.

Examples

Give all drinkers who do not like "Guinness"

- Likes(Drinker, Beer)
- Serves(Pub, Beer)
- Visits(Drinker, Pub)



Examples

- Likes(Drinker, Beer)
- Serves(Pub, Beer)
- Visits(Drinker, Pub)

Give all drinkers who visit a bar that serves "Guinness"

$$\pi_{\text{Drinker}} \sigma_{\text{GPub=Pub}} (\text{Visits} \times (\rho_{\text{GPub/Pub}} \sigma_{\text{Beer="Guinness"}} \text{Serves}))$$

Give all drinkers who visit a bar that serves a beer they like

$$\begin{aligned} & \pi_{\text{Drinker}} \sigma_{\text{VP=SP} \wedge \text{SB=Beer} \wedge \text{Drinker=VD}} \\ & (\text{Likes} \times \rho_{\text{SP/Pub, SB/Beer}} \text{Serves} \times \rho_{\text{VD/Drinker, VP/Pub}} \text{Visits}) \\ = & \\ & \pi_{\text{Drinker}} \sigma_{\text{SB=Beer} \wedge \text{Drinker=VD}} [\text{Likes} \times \\ & \sigma_{\text{VP=SP}} (\rho_{\text{SP/Pub, SB/Beer}} \text{Serves} \times \rho_{\text{VD/Drinker, VP/Pub}} \text{Visits})] \end{aligned}$$

Keys & Functional Dependencies

A set of one or more attributes $\{A_1, A_2, \dots, A_n\}$ is called a **key** for a relation R if:

- (1) $\{A_1, A_2, \dots, A_n\}$ functionally determines *all attributes* of R .
- (2) *No proper subset* of $\{A_1, A_2, \dots, A_n\}$ functionally determines all attributes of R .

→ In other words, a key must be a **minimal** set of attributes for which property (1) holds.

A set of attributes $\{A_1, A_2, \dots, A_n\}$ that contains a key of relation R is called a **superkey** of R .

→ Every key is also a superkey.

A Complete Set of Inference Rules

Armstrong's Axioms:

The following three axioms allow for deriving any FD that follow from a given set of FD's.

1. Reflexivity:

If $\{B_1, B_2, \dots, B_m\} \subseteq \{A_1, A_2, \dots, A_n\}$,
then $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ ($\rightarrow$ "trivial FDs")

2. Augmentation:

If $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$,
then $A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k \rightarrow B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k$,
for any set of attributes $C_1, C_2, \dots, C_k$ ($\rightarrow$ "non-trivial FDs")

3. Transitivity:

If $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ and $B_1, B_2, \dots, B_m \rightarrow C_1, C_2, \dots, C_k$,
then $A_1, A_2, \dots, A_n \rightarrow C_1, C_2, \dots, C_k$ ($\rightarrow$ "closure algorithm")

Normal Forms of Relations

In the following, let X be a set of attributes and Y be a single attribute of a relation R , and let F be a given set of FD's on R .

R is in **Third Normal Form** (3NF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R or Y is an attribute of a key of R .

R is in **Boyce-Codd Normal Form** (BCNF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R .

Algorithm for Computing a Dependency-Preserving Decomposition in 3NF

Given a relation R with attributes A , a set of keys K , and a *minimal basis* of FD's F .

Decompose R into:

- $R_1 = \pi_{K'}(R)$ where K' is a key in K which becomes key of R_1 ; (if there are multiple keys in R , choose one)
- for each $X \rightarrow Y$ in F , the projection $R_{i+1} = \pi_{X \cup Y}(R)$ forms a new relation in the decomposition with key X .



Algorithm for Computing a Decomposition in BCNF (not dependency-preserving!)

Given a relation R with attributes A , a set of keys K , and a set of FD's F .

If there is a non-trivial FD $X \rightarrow Y$, where X is not a superkey of R , then decompose R into:

- $S = \pi_{X \rightarrow Y}(R)$ with the new key X ;
- $T = \pi_{X \cup (A - X \rightarrow Y)}(R)$ by taking over all remaining keys from R .

Then recursively decompose S and T into BCNF using the remaining (non-violated) FD's in F .



Multivalued Dependency

A **multivalued dependency** (MVD), denoted as

$$A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$$

holds for a relation R if, for each pair of tuples t, u of R , which agree on all the A 's, there is a tuple v in R that also agrees

- with both t and u on the A 's;
- with t on the B 's;
- with u on *all other attributes* of R not among the A 's and B 's.

Example:

	starName A's	city,street B's	title,year others
t	a ₁	b ₁	c ₁
	Ford	Malibu, Maple Str.	Star Wars, 1977
v	a ₁	b ₁	c ₂
	Ford	Malibu, Maple Str.	Star Wars, 1980
u	a ₁	b ₂	c ₂
	Ford	Hollywood, Mulh. Dr.	Star Wars, 1980



Example: Arithmetic Predicates

- A beer is “cheap” if there are at least two different bars that sell it for under \$2.

Cheap(beer) :-

Sells(bar1,beer,p1) AND Sells(bar2,beer,p2)

AND p1 < 2.00 AND p2 < 2.00

AND bar1 <> bar2



Abbreviated Syntax

EXAMPLES

- **/doc/chapter[5]/section[2]**
/child::doc/child::chapter[position()=5]
/child::section[position()=2]
- **chapter//a** is an abbreviation of
chapter/descendant-or-self::node()/child::a
- **//a** is an abbreviation of
/descendant-or-self::node()/child::a
- **/a//.** is an abbreviation of
/child::a/descendant-or-self::node()/self::node()
- **chapter/.a** is an abbreviation of
child::chapter/self::node()/child::a